OCaml User-defined Types & Testing

Jibesh Patra, Soumyajit Dey

Week 4 – Software Engineering – CS20202 – Spring 2026 – IIT Kharagpur

User-defined Types Cont.

Record Type

- A composite of other types of data, each of which is named.
- Elements in Lists and Tuples are ordered by the sequence.
- Records names the elements providing each with a unique label.

Record Type Example

```
type shape =
 | Circle of float
 | Point
 | Triangle of float * float

type color =
| Red
| Green
| Blue
```

```
type shapes = {
  shape_color: color;
  shape: shape;
  id: int
}
```

A record type with three *fields*

```
let s1 = {id= 23; shape_color= Red; shape=Circle 2.5}
let s2 = {id= 42; shape_color= Green; shape=Point}
let sh = s1.shape
```

field access

```
..
type shapes = { shape_color : color; shape : shape; id : int; }
val s1 : shapes = {shape_color = Red; shape = Circle 2.5; id = 23}
val s2 : shapes = {shape_color = Green; shape = Point; id = 42}
val sh : shape = Circle 2.5
```

Q4.1 Which of the following expressions will type-check as a value of type `point`?

```
type point = {
  x : int;
  y : int;
}
```

1. `{ x = 3; y = 4 }`
2. `{ y = 4; x = 3 }`
3. `(3, 4)`
4. `{ x = 3 }`

Pattern Matching – Records

```
let show_shape sh = match sh with
| {id=i; shape=s; shape_color=sc} -> s;;

show_shape s1;;
```

```
..
val show_shape : shapes -> shape = <fun>
- : shape = Circle 2.5
```

Explicitly mapping each field to a new variable name.
Avoids variable name collisions.

Punning: Most common

```
let id = 100_000
let show_shape sh = match sh with
| {id; shape; shape_color} -> id;;
show_shape s1;;
```

```
..
val id : int = 100000
val show_shape : shapes -> int = <fun>
- : int = 100000
```

Pattern Matching – Records

```
let show_shape sh = match sh with
| {shape; _} -> shape;;

show_shape s1;;
```

```
let show_shape {shape; _} = shape;;
show_shape s1;;
```

```
..
val show_shape : shapes -> shape = <fun>
- : shape = Circle 2.5
```

Pattern Matching – Records (Conditions)

```
let s1 = {id= 23; shape_color= Red; shape=Circle 2.5}
let s2 = {id= 42; shape_color= Green; shape=Point}

let is_correct_id sh = match sh with
| {id=42; _} -> true
| _ -> false


let a = is_correct_id s1
let b = is_correct_id s2
```

```
..
val is_correct_id : shapes -> bool = <fun>
val b : bool = false
val c : bool = true
```

Pattern Matching – Records (Conditions)

```
let s1 = {id= 23; shape_color= Red; shape=Circle 2.5}
let s2 = {id= 42; shape_color= Green; shape=Point}
let s3 = {id= 22; shape_color= Blue; shape=Triangle (2.4, 3.5)}
```

The return code is executed only if the pattern matches and the condition in the `when` clause is satisfied.

```
let is_red_or_id_more sh =
  match sh with
  | {shape_color = Red;_} -> true
  | {id;_} when id > 30 -> true
  | _ -> false
```

```
..
val is_red_or_id_more : shapes -> bool = <fun>
val check1 : bool = true
val check2 : bool = true
val check3 : bool = false
```

```
let check1 = is_red_or_id_more s1
let check2 = is_red_or_id_more s2
let check3 = is_red_or_id_more s3
```

Aliases

- Aliases allow to give a name to a type.

```
type coordinate = float * float;;
let x_y: coordinate = (1.2, 2.3);;
```

```
type coordinate = float * float
val x_y : coordinate = (1.2, 2.3)
```

Giving names to function parameters

```
let sum_nums (x, y) = x + y;;
let foo ((x, y) as arg) = sum_nums arg;;
foo (1,2);;
```

```
val sum_nums : int * int -> int = <fun>
val foo : int * int -> int = <fun>
- : int = 3
```

Checking Correctness

Checking Correctness of Code

- How to know our programs are correct?
- Formal verification: a mathematical proof that the code does what it's supposed to do. A guarantee that the code generates the appropriate values regardless of what inputs it is given.
- Testing: verify that the code generates the appropriate values on some of the inputs if proving for all values is difficult.

difficult to provide formal specifications.

Testing Pitfall

```
let abs x =
  if x > 0 then x else -x;;

abs 5;;
abs 1;;
abs (-5);;
abs 0;;
abs min_int;;
```

```
val abs : int -> int = <fun>
- : int = 5
- : int = 1
- : int = 5
- : int = 0
- : int = -4611686018427387904
```

Unit Testing

```
let add x y = x + y;;
let add_test() = add 2 3 = 5 && add 10 2 = 12 && add 50 50 = 100;;
add_test();;
```

Function we want to test

Unit testing function that verifies add calculates the correct values

```
val add : int -> int -> int = <fun>
val add_test : unit -> bool = <fun>
- : bool = true
```

Unit Testing

```
let unit_test_helper (test : bool) (msg : string) : unit = if test then Printf.printf "%s
PASS\n" msg else Printf.printf "%s FAIL\n" msg;;

let add x y = x + y;;

let add_test () =
 unit_test_helper (add 2 3 = 5) "Adding 2, 3";
 unit_test_helper (add 4 3 = 7) "Adding 4, 3";
 unit_test_helper (add 1 3 = 4) "Adding 1, 3";
 unit_test_helper (add 20 30 = 5) "Adding 20, 30";;

add_test();;
```

```
…
Adding 2, 3 PASS
Adding 4, 3 PASS
Adding 1, 3 PASS
Adding 20, 30 FAIL
- : unit = ()
```

Sequencing operator ; allows chaining of expressions

Standard Library

Standard Library

- There exists a wide range of built-in functions in OCaml.
- The functions are divided into *modules*.
- List of all modules link.
- Examples of popular modules:
  - `List` -> Go through the documentation
  - `Array` -> Go through the documentation
  - `Char` -> Go through the documentation
  - `String` -> Go through the documentation
  - `Random` -> Go through documentation
  - `Buffer` -> Go through the documentation
  - `Printf` -> Go through the documentation

Examples – List and Array

```
let l = List.length [1;2;3]
let n_elem = List.nth ["a"; "b"; "c"; "d"] 3
```

```
val l : int = 3
val n_elem : string = "d"
```

```
let a = [| 10; 20; 30 |];;
Array.length a;;
```

```
val a : int array = [|10; 20; 30|]
- : int = 3
```

Examples – String and Sys

```
String.length "hello";;
String.uppercase_ascii "software engineering";;
String.concat ", " ["a"; "b"; "c"];;
```

```
- : int = 5
- : string = "SOFTWARE ENGINEERING"
- : string = "a, b, c"
```

```
let curdir = Sys.getcwd ()
let v = Sys.argv
```

Examples – Random

```
let rand_int = Random.int 10
let rand_float = Random.float 1.0
```

```
val rand_int : int = 8
val rand_float : float = 0.222980424880495964
```

Example – Random

```
let _ = Random.init 42
let limit = 100
let r1 = Random.int limit
let r2 = Random.int limit

let r3 =  Random.int limit
let r4 = Random.int limit
let r5 = Random.int limit

let _ =  Random.init 42
let r6 = Random.int limit
let r7 = Random.int limit
let r8 = Random.int limit
let r9 = Random.int limit
```

```
let _ =  Random.init 10
let r10 = Random.int limit
let r11 = Random.int limit
let r12 = Random.int limit

Random.self_init ();;
```

```
- : unit = ()
val limit : int = 100
val r1 : int = 64
val r2 : int = 20
val r3 : int = 47
val r4 : int = 96
val r5 : int = 51
- : unit = ()
val r6 : int = 64
val r7 : int = 20
val r8 : int = 47
val r9 : int = 96
- : unit = ()
val r10 : int = 67
val r11 : int = 22
val r12 : int = 37
```

Example – Random & List

```
let _ = Random.init 42
let random_float_list n =
  List.init n (fun _ -> Random.float 2.0)
let floats = random_float_list 3
```

```
- : unit = ()
val random_float_list : int -> float list =
<fun>
val floats : float list =
  [0.88952958105468638; 1.55070718917561923;
0.7465935198927176O5]
```

```
let _ = Random.self_init ()
let random_choice lst =
  List.nth lst (Random.int (List.length lst))
let random_car_list n =
  List.init n (fun _ -> random_choice ["honda"; "tata"; "mahindra"; "maruti"])
let car = random_car_list 1
```

```
- : unit = ()
val random_choice : 'a list -> 'a = <fun>
val random_car_list : int -> string list = <fun>
val car : string list = ["mahindra"]
```

Error Handling

Exceptions

- We might occasionally encounter anomalous conditions in programs.
- Mechanisms in OCaml:
  - Option
  - Exception
- Problem with Option: We need to unwrap the values.

Exception Example

```
let first_elem_of_list lst : int = match
lst with
| [] -> raise Exit
| h::_ -> h

let f = first_elem_of_list [1; 2; 3]
let v = first_elem_of_list []
```

Should be of type `exn` (Exception)

```
val first_elem_of_list : int list -> int = <fun>
val f : int = 1
Exception: Stdlib.Exit.
```

`Stdlib` library module

Exception Example

- raise Invalid_argument
- raise Failure
- Custom

```
exception Empty_list_error
let first_elem_of_list lst : int = match lst with
| [] -> raise Empty_list_error
| h::_ -> h
let v = first_elem_of_list []
let f = first_elem_of_list [1; 2; 3]
```

```
exception Empty_list_error
val first_elem_of_list : int list -> int = <fun>
Exception: Empty_list_error.
```

Custom Exception with Data

```
exception Not_Found of int
let get_html_text req = match req with
| "GET" -> "<html>Text</html>"
| _ -> raise (Not_Found 404)

let content = get_html_text "GET"
let content = get_html_text "PUT"
```

```
exception Not_Found of int
val get_html_text : string -> string = <fun>
val content : string = "<html>Text</html>"
Exception: Not_Found 404.
```

Pattern Matching – Exceptions

```
exception Not_Found of int
let get_html_text req = match req with
| "GET" -> "<html>Text</html>"
| "CUT" -> raise Exit
| "ABCD" -> raise (Failure "Failed here")
| _ -> raise (Not_Found 404)
```

```
let find_the_exception f =
 try
  f ()
 with
 | Not_Found 404 -> "HTML not found error"
 | Failure _ -> "HTML Failed"
 | Exit -> "HTML Exit"
```

```
let no_error = find_the_exception ( fun () -> get_html_text "GET")
let er3 = find_the_exception ( fun () ->  get_html_text "XY")
let er1 = find_the_exception ( fun () -> get_html_text "CUT" )
let er2 = find_the_exception ( fun () ->  get_html_text "ABCD" )
```

Summary of Data Types

Type	Type Constructor	Value Constructor
functions	`<> -> <>`	`fun <> -> <>`

tuples	`<> * <>`
	`<> * <> * <>`
		`<>, <>`
		`<>, <>, <>`
		…

lists	`<> list`	`[]`
		`<>::<>`
		`[<>; <>; …]`

records	`{<>:<>; <>:<>; …}`	`{<>=<>; <>=<>; …}`
options	`<> option`	`None` `Some <>`

exceptions	`exn`	`Exit`
		`Failure <>`
		…

Exercise

- Write a function to approximate the value of *pi* using the Monte-Carlo method. The function should take `‘n’` as input for generating `‘n’` points at random. The following are some example input outputs:
  - If n -> 5 the value of pi is 4.000000
  - If n -> 20 the value of pi is 3.00000
  - If n -> 1000 the value of pi is 3.114000
- Are the following functions well typed? Write the `function type` otherwise write why the function is not well typed.
  - `let f x =  if x then 1 else ""`
  - `let foo lst = match lst with [] -> [] | y :: s -> [s]`

Higher-Order Programming

Advantages of Higher-Order Programming

- In OCaml, functions are values
  - can be passed around as arguments to other functions just like integers, strings, or any other value.
  - can be stored in data structures (such as lists, records, or tuples).
  - can be returned as results from other functions, enabling higher-order and compositional programming.
- Higher-order functions satisfy the *edict of irredundancy*.
- Well known abstractions:
  - *map* → performing an operation on all elements of a list.
  - *fold* → combining all of the elements of a list one at a time with a binary function, starting with an initial value.
  - *filter* → returning a subset of elements of a list based on certain conditions.

Map Abstraction

```
let rec incr_elements (inp : int list) : int list =
match inp with
| [] -> []
| head :: tail -> (1 + head) :: (incr_elements tail)
```

```
fun x -> 1 + x
```

```
let rec sqr_elements (inp: int list): int list =
 match inp with
| [] -> []
| head :: tail -> (head * head) :: (sqr_elements tail)
```

```
fun x -> x * x
```

```
let inc_vals = incr_elements [1; 2; 3]
let sq_vals = sqr_elements [1; 2; 3]
```

Map Abstraction

```
let rec map (f : int -> int) (inp : int list) : int list =
match inp with
| [] -> []
| head :: tail -> f head :: (map f tail)

let inc_vals = map (fun x -> x + 1) [1; 2; 3]
let sq_vals = map (fun x -> x * x) [1; 2; 3]
```

```
val map : (int -> int) -> int list -> int list = <fun>
val inc_vals : int list = [2; 3; 4]
val sq_vals : int list = [1; 4; 9]
```

Fold Abstraction

```
let rec sum (inp : int list) : int =
match inp with
| [] -> 0
| head :: tail -> head + (sum tail)

let rec prod (inp : int list) : int =
match inp with
| [] -> 1
| head :: tail -> head * (prod tail)
```

```
let sum_elms = sum [1; 2; 3; 4]
let prod_elms = prod [1; 2; 3; 4]
```

Fold Abstraction

```
let rec fold (f : int -> int -> int)
(inp : int list)
(acc : int)
: int =
match inp with
| [] -> acc
| hd :: tl -> f hd (fold f tl acc)

let sum_elms = fold (fun x y -> x + y) [1; 2; 3; 4] 0
let prod_elms = fold (fun x y -> x * y) [1; 2; 3; 4] 1

let sum_elms = fold (+) [1; 2; 3; 4] 0
let prod_elms = fold ( * ) [1; 2; 3; 4] 1
```

Filter Abstraction

```
let rec evens inp =
match inp with
| [] -> []
| head :: tail -> if head mod 2 = 0 then head :: evens tail
else evens tail

let rec odds inp =
match inp with
| [] -> []
| head :: tail -> if head mod 2 <> 0 then head :: odds tail
else odds tail
```

```
let even_nums = evens [1; 2; 3; 4]
let odd_nums = odds [1; 2; 3; 4]
```

Filter Abstraction

```
let rec filter predicate list =
 match list with
 | [] -> []
 | head :: tail ->
     if predicate head then
       head :: filter predicate tail
     else
       filter predicate tail
```

```
let even_nums = filter (fun x -> x mod 2 = 0) [1; 2; 3; 4]
let odd_nums = filter (fun x -> x mod 2 <> 0) [1; 2; 3; 4]
```

Modules

Module Systems

- Namespaces
- Abstraction
- Code reuse

General Form of Modules

```
struct
    ⟨definition1⟩
    ⟨definition2⟩
    ⟨definition3⟩
    …
end
```

Module Example

stack.ml🐫

```
module ListStack = struct
  let empty = []
  let is_empty = function [] -> true | _ -> false
  let push x s = x :: s
  exception Empty
  let peek = function
    | [] -> raise Empty
    | x :: _ -> x
  let pop = function
    | [] -> raise Empty
    | _ :: s -> s
end
```

test.ml🐫

```
open ListStack
let check = is_empty []
```

```
utop # #use "stack.ml";;
utop # #use "test.ml";;
val check : bool = true
```

Ref: https://cs3110.github.io/textbook/chapters/modules/modules.html

This is the end

CC BY SA